Atelier 4 :Les Curseurs en MySQL – Cours Pratique

1. Qu'est-ce qu'un curseur ?

En MySQL, un **curseur** est un outil qui permet de **parcourir les résultats d'une requête ligne par ligne**.
C'est un peu comme un "pointeur" ou un **lecteur séquentiel** qui lit chaque enregistrement d'une table, un à un.

 Pourquoi les utiliser ?
Les curseurs sont utiles lorsque vous devez :

- effectuer des traitements **personnalisés sur chaque ligne** d'un jeu de résultats ;
- appliquer une logique **conditionnelle** ou **arithmétique** à chaque enregistrement ;
- automatiser des tâches répétitives dans une **procédure stockée**.

 Exemple d'utilisation typique : calculer un total, mettre à jour une colonne, générer des factures, etc.

---

2. Étapes de manipulation d'un curseur

L'utilisation d'un curseur suit toujours **le même cycle en quatre étapes** :
Déclarer → Ouvrir → Lire → Fermer

1- Déclaration du curseur

Le curseur doit être déclaré **dans une procédure**, à l'intérieur d'un bloc `BEGIN...END`.

```
DECLARE nom_curseur CURSOR FOR
SELECT colonne1, colonne2 FROM table;
```

2- Ouverture du curseur

Avant de parcourir les résultats, il faut l'ouvrir :

```
OPEN nom_curseur;
```

3-Lecture (FETCH)

On récupère les valeurs ligne par ligne dans des variables :

```
FETCH nom_curseur INTO variable1, variable2;
```

4-Fermeture du curseur

Quand la lecture est terminée, il faut libérer les ressources :

```
CLOSE nom_curseur;
```

---

3. Gestion de la fin de lecture

Pour éviter que la lecture continue après la dernière ligne, on définit un **gestionnaire de fin** :

```
DECLARE fin_curseur BOOLEAN DEFAULT FALSE;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_curseur = TRUE;
```

Cela signifie : “Quand il n’y a plus de ligne à lire, mets `fin_curseur` à `TRUE` et continue le programme sans erreur.”

---

4. Exemple : Boucle avec curseur

Voici un modèle général de boucle :

```
DELIMITER /
CREATE PROCEDURE exemple_curseur()
BEGIN
    DECLARE var1 INT;
    DECLARE var2 VARCHAR(100);
    DECLARE fin_curseur BOOLEAN DEFAULT FALSE;

    DECLARE mon_curseur CURSOR FOR
        SELECT id, nom FROM stagiaires;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_curseur = TRUE;

    OPEN mon_curseur;

    boucle_lecture: LOOP
        FETCH mon_curseur INTO var1, var2;
        IF fin_curseur THEN
            LEAVE boucle_lecture;
        END IF;

        -- Traitement sur chaque ligne
        SELECT CONCAT('Lecture du stagiaire : ', var2) AS message;
    END LOOP;

    CLOSE mon_curseur;
END /
DELIMITER ;
```

LOOP : C'est l'instruction qui initialise la boucle. Elle indique que le code contenu dans le bloc de la boucle doit être exécuté de manière répétitive.

 Explication :

- Le curseur lit chaque stagiaire.
- La boucle `LOOP` continue jusqu'à la fin du jeu de résultats.
- `LEAVE` permet de sortir de la boucle proprement.

---

5. Cas Pratique – Calcul de facturation avec curseur

Imaginons que vous devez calculer le **montant total d'une facture d'eau** en fonction de la consommation (`qte_eau`).

Étape 1 : Création de la table

```
CREATE TABLE IF NOT EXISTS facture (
    id_facture INT AUTO_INCREMENT PRIMARY KEY,
    nom_client VARCHAR(100),
    qte_eau INT,
    montant DECIMAL(10, 2)
);
```

Étape 2 : Insertion de données de test

```
INSERT INTO facture (nom_client, qte_eau) VALUES
('Client 1', 9),
('Client 2', 15),
('Client 3', 25);
```

Étape 3 : Création d'une procédure avec curseur

Cette procédure parcourt toutes les factures, calcule le montant et met à jour la colonne `montant`.

```
DELIMITER /
CREATE PROCEDURE calculer_factures()
BEGIN
    DECLARE v_id INT;
    DECLARE v_qte INT;
    DECLARE v_montant DECIMAL(10,2);
    DECLARE fin BOOLEAN DEFAULT FALSE;

    DECLARE curseur_facture CURSOR FOR
        SELECT id_facture, qte_eau FROM facture;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;

    OPEN curseur_facture;

    boucle_factures: LOOP
        FETCH curseur_facture INTO v_id, v_qte;
        IF fin THEN
            LEAVE boucle_factures;
        END IF;

        -- Calcul du montant selon une règle simple
        SET v_montant = v_qte * 1.25;

        -- Mise à jour du montant dans la table
        UPDATE facture SET montant = v_montant WHERE id_facture =
v_id;
    END LOOP;
```

```
    CLOSE curseur_facture;
END /
DELIMITER ;
```

✅Étape 4 : Exécution

```
CALL calculer_factures();
SELECT * FROM facture;
```

Résultat attendu :
Chaque ligne de la table `facture` aura désormais un montant calculé automatiquement selon la quantité d'eau consommée.

---

6. Ce qu'il faut retenir

Étape	Action	Commande clé
Déclaration	Préparer le curseur	`DECLARE ... CURSOR FOR SELECT ...`
Ouverture	Démarrer la lecture	`OPEN nom_curseur;`
Lecture	Récupérer les valeurs	`FETCH ... INTO ...;`
Fermeture	Libérer les ressources	`CLOSE nom_curseur;`
Fin de lecture	Gérer la fin du jeu	`DECLARE HANDLER FOR NOT FOUND ...`

---

7. Exercices pour les stagiaires

1. Créez une table `notes` avec les colonnes : `id_note`, `nom_etudiant`, `note`, `mention`.
2. Remplissez-la avec quelques données.
3. Créez une **procédure avec curseur** qui :
   - parcourt chaque étudiant,
   - attribue une mention (`Excellent`, `Bien`, `Passable`, `Insuffisant`) selon la note,
   - met à jour la colonne `mention`.
4. Vérifiez le résultat final avec `SELECT * FROM notes;`